

TALLER PROGCOMP: TRACK OPTIMIZACIÓN

PROGRAMACIÓN DINÁMICA II

Gabriel Carmona Tabja

Universidad Técnica Federico Santa María,
Università di Pisa

April 12, 2026

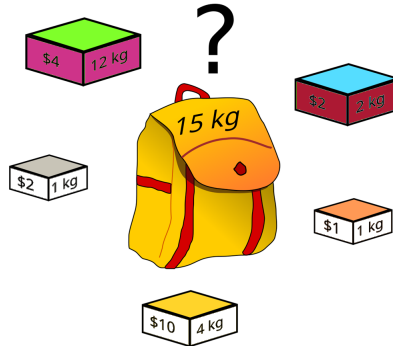
Part I

PROBLEMA DE LA MOCHILA

PROBLEMA DE LA MOCHILA

Definición

Tengo n objetos, donde cada objeto tiene un valor y un peso. Tienes una mochila que soporta un peso máximo de P . Determinar cuanto es el valor máximo que puedes llevar en la mochila sin sobrepasar el peso.



EJEMPLO

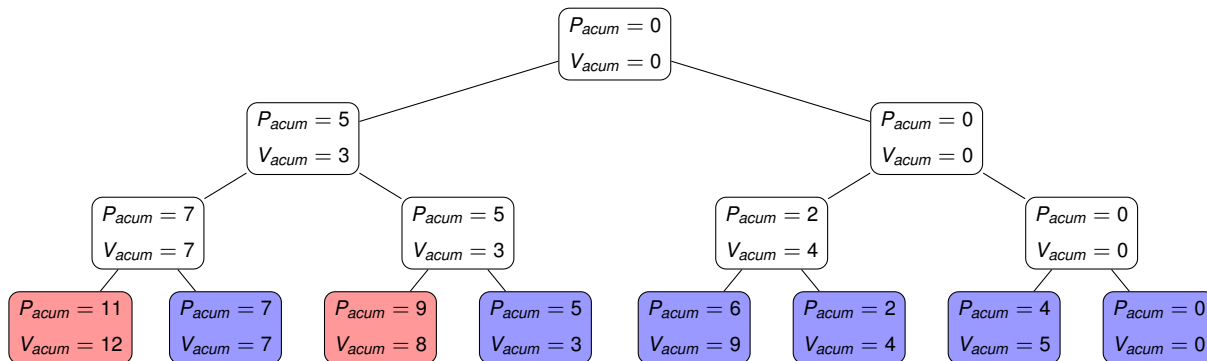
Se tiene el siguiente ejemplo $P = 7$ y los objetos son:

$$O = \{\{v : 3, p : 5\}, \{v : 4, p : 2\}, \{v : 5, p : 4\}\}$$

EJEMPLO

Se tiene el siguiente ejemplo $P = 7$ y los objetos son:

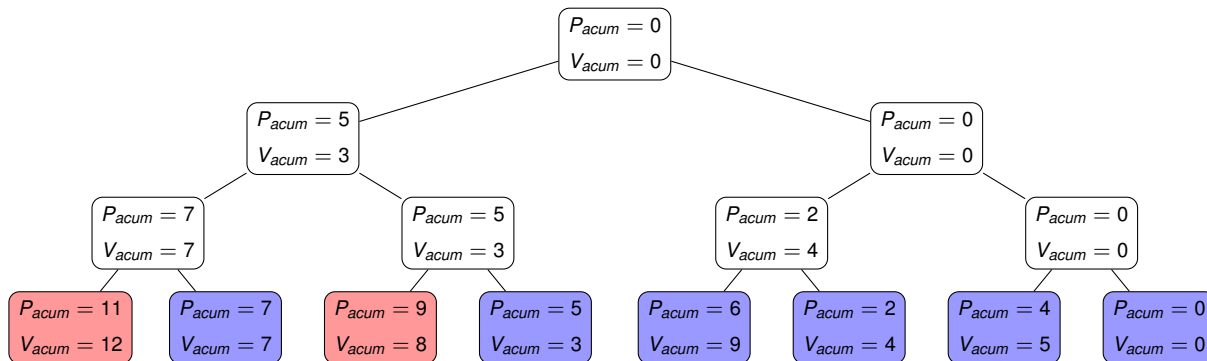
$O = \{\{v : 3, p : 5\}, \{v : 4, p : 2\}, \{v : 5, p : 4\}\}$



EJEMPLO

Se tiene el siguiente ejemplo $P = 7$ y los objetos son:

$O = \{\{v : 3, p : 5\}, \{v : 4, p : 2\}, \{v : 5, p : 4\}\}$



La respuesta sería entonces guardar el objeto 2 y el objeto 3

CÓDIGO RECURSIVO

```
1
2 int mochila(int W, int n, vector< pair< int, int > > &objs, int obj_act, int val_act, int
   peso_act){
3     if (W < peso_act) return -1;
4
5     if (n == n_actual){
6         return val_act;
7     }
8
9
10    // se considera el objeto
11    int si =
12        mochila(W, n, objs,
13            obj_act + 1, val_act + objs[obj_act].second, peso_act + objs[obj_act].first);
14
15    // no se considera el objeto
16    int no =
17        mochila(W, n, objs,
18            obj_act + 1, val_act, peso_act);
19    return max(si, no);
20 }
```

CÓDIGO RECURSIVO

```
1
2 int mochila(int W, int n, vector< pair< int, int > > &objs, int obj_act, int val_act, int
   peso_act){
3     if (W < peso_act) return -1;
4
5     if (n == n_actual){
6         return val_act;
7     }
8
9
10    // se considera el objeto
11    int si =
12        mochila(W, n, objs,
13            obj_act + 1, val_act + objs[obj_act].second, peso_act + objs[obj_act].first);
14
15    // no se considera el objeto
16    int no =
17        mochila(W, n, objs,
18            obj_act + 1, val_act, peso_act);
19    return max(si, no);
20 }
```

¿Cuál es la complejidad temporal de este código?

CÓDIGO RECURSIVO

```
1
2 int mochila(int W, int n, vector< pair< int, int > > &objs, int obj_act, int val_act, int
   peso_act){
3     if (W < peso_act) return -1;
4
5     if (n == n_actual){
6         return val_act;
7     }
8
9
10    // se considera el objeto
11    int si =
12        mochila(W, n, objs,
13            obj_act + 1, val_act + objs[obj_act].second, peso_act + objs[obj_act].first);
14
15    // no se considera el objeto
16    int no =
17        mochila(W, n, objs,
18            obj_act + 1, val_act, peso_act);
19    return max(si, no);
20 }
```

¿Cuál es la complejidad temporal de este código?

$O(2^n)$

PROBLEMA DE LA MOCHILA - RE-FORMULACIÓN

$$mochila(i, w) = \begin{cases} 0, & \text{si } i = 0 \text{ o } w = 0 \\ mochila(i - 1, w), & \text{si } w_i > w \\ \max\{mochila(i - 1, w), v_i + mochila(i - 1, w - w_i)\}, & \text{si } w_i \leq w \end{cases}$$

PROGRAMACIÓN DINÁMICA

$$P = 7 \text{ y } O = \{\{v : 3, p : 5\}, \{v : 4, p : 2\}, \{v : 5, p : 4\}\}$$

$i \backslash w$	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1 (5,3)	0	0	0	0	0	0	0	0
2 (2,4)	0	0	0	0	0	0	0	0
3 (4,5)	0	0	0	0	0	0	0	0

PROGRAMACIÓN DINÁMICA

$$P = 7 \text{ y } O = \{\{v : 3, p : 5\}, \{v : 4, p : 2\}, \{v : 5, p : 4\}\}$$

$i \backslash w$	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1 (5,3)	0	0	0	0	0	3	3	3
2 (2,4)	0	0	0	0	0	0	0	0
3 (4,5)	0	0	0	0	0	0	0	0

PROGRAMACIÓN DINÁMICA

$$P = 7 \text{ y } O = \{\{v : 3, p : 5\}, \{v : 4, p : 2\}, \{v : 5, p : 4\}\}$$

$i \backslash w$	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1 (5,3)	0	0	0	0	0	3	3	3
2 (2,4)	0	0	4	4	4	4	4	7
3 (4,5)	0	0	0	0	0	0	0	0

PROGRAMACIÓN DINÁMICA

$$P = 7 \text{ y } O = \{\{v : 3, p : 5\}, \{v : 4, p : 2\}, \{v : 5, p : 4\}\}$$

$i \backslash w$	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1 (5,3)	0	0	0	0	0	3	3	3
2 (2,4)	0	0	4	4	4	4	4	7
3 (4,5)	0	0	4	4	5	5	9	9

CÓDIGO

```
1
2 // se asume una matriz m[n + 1][W + 1]
3 // se asume un arreglo de objs[n],
4 //     donde first guarda el peso
5 //     y second guarda el valor
6 int mochila(int W, int n, vector< pair< int, int > > &objs){
7     int i, w;
8
9     vector< vector< int > > m(n + 1, vector< int >(W + 1, 0));
10    for(i = 0; i <= n; i++){
11        for(w = 0; w <= W; w++){
12            if(i == 0 || w == 0)
13                // casos base / borde
14                m[i][w] = 0;
15            else if(objs.first[i - 1] <= w)
16                m[i][w] = max(objs.second[i - 1] + m[i - 1][w - objs.first[i - 1]], m[i - 1][w]);
17            else
18                m[i][w] = m[i - 1][w];
19        }
20    }
21    return m[n][W];
22 }
```

¿Cuál es la complejidad temporal?

CÓDIGO

```
1
2 // se asume una matriz m[n + 1][W + 1]
3 // se asume un arreglo de objs[n],
4 //     donde first guarda el peso
5 //     y second guarda el valor
6 int mochila(int W, int n, vector< pair< int, int > > &objs){
7     int i, w;
8
9     vector< vector< int > > m(n + 1, vector< int >(W + 1, 0));
10    for(i = 0; i <= n; i++){
11        for(w = 0; w <= W; w++){
12            if(i == 0 || w == 0)
13                // casos base / borde
14                m[i][w] = 0;
15            else if(objs.first[i - 1] <= w)
16                m[i][w] = max(objs.second[i - 1] + m[i - 1][w - objs.first[i - 1]], m[i - 1][w]);
17            else
18                m[i][w] = m[i - 1][w];
19        }
20    }
21    return m[n][W];
22 }
```

¿Cuál es la complejidad temporal?

$O(nW)$ (pseudo-polinomial)

REFERENCES I